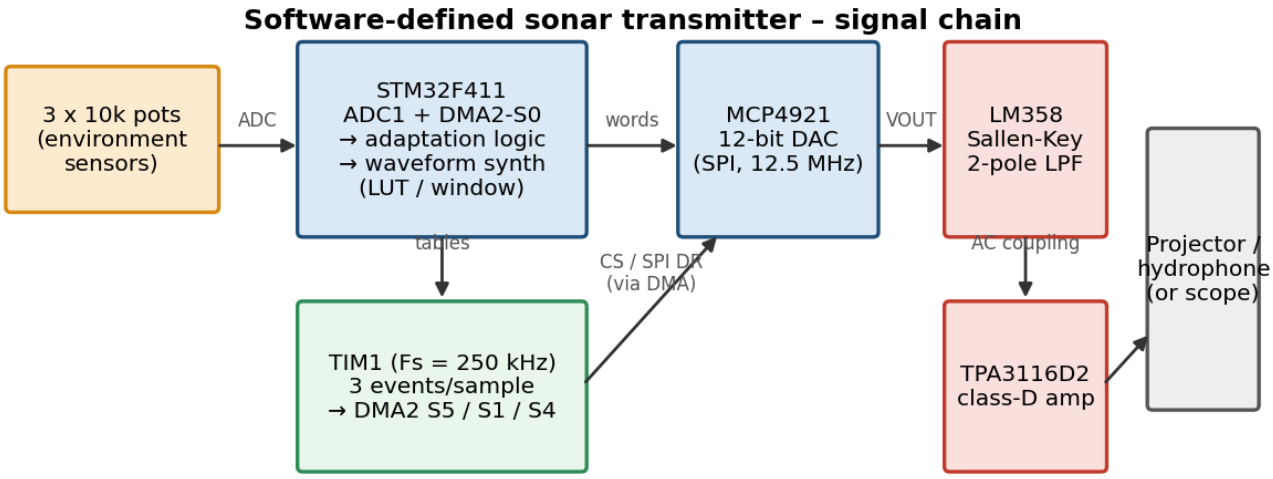
Software-Defined Sonar Transmitter

Mathematics, DMA / Timer Hardware and Signal-Chain Study Guide

STM32F411 "Black Pill" | MCP4921 12-bit DAC | LM358 Sallen-Key filter | 3 potentiometer "sensors"



How to use this guide. Part A explains *why* the system exists (sonar physics and what the three adaptable parameters do). Part B is the signal mathematics (LFM, geometric, phase-coded, windows). Part C covers the digital-to-analog chain (sampling, DAC, filter). Part D explains the timer / DMA / ADC machinery that makes the firmware low-power. Part E gives the power budget logic, how to validate with an FFT, and a self-test question bank. Every plot in this guide that shows a transmitted pulse was produced by running the **actual firmware synthesis code** (SonarTx.ino) on a PC, so what you see is what your DAC is fed.

Contents

Contents 2

1. Why a "software-defined" sonar? The physics in one page 4

1.1 Range from time, resolution from bandwidth . . . 4

1.2 Absorption: why frequency must be chosen from the water . . . 4

1.3 The sonar equation (how energy, noise and range tie together) . . . 4

1.4 What each pot means physically . . . 4

2. Linear Frequency Modulation (LFM) - the core mathematics 6

2.1 Definition . . . 6

2.2 Time-bandwidth product and spectrum . . . 6

2.3 Matched filter and pulse compression . . . 6

2.4 Doppler and ambiguity . . . 7

3. Other modulations in the firmware 8

3.1 Geometric (exponential) sweep . . . 8

3.2 Phase-coded pulse (BPSK) . . . 8

4. Windowing (apodisation) 10

5. Sampling, quantisation and the DAC 12

5.1 Sample rate choice . . . 12

5.2 Quantisation noise . . . 12

5.3 Zero-order hold and images . . . 12

5.4 DDS view and the sine look-up table . . . 12

6. The analog front-end: Sallen-Key low-pass (LM358) 13

6.1 Why it is needed . . . 13

6.2 Unity-gain Sallen-Key (2nd order) . . . 13

6.3 LM358 on a single 5 V supply - practical limits . . . 13

7. Timer + DMA architecture (the "hardware-level optimisation") 14

7.1 What DMA is . . . 14

7.2 Why one stream is not enough: the MCP4921 chip-select problem . . . 14

7.3 Timing arithmetic and bus load . . . 15

7.4 Ping-pong scheduling (what the CPU does and when) . . . 15

8. Reading the "sensors": ADC + DMA 16

9. Energy, power and battery 16

10. Validating the output with an oscilloscope FFT 17

What to expect . . . 17

Debug checklist . . . 17

11. Code-to-math map 17

12. Self-test question bank 18

Answers . . . 18

13. Further reading and datasheets **18**

1. Why a "software-defined" sonar? The physics in one page

A side-scan sonar sends an acoustic pulse ("ping"), listens for echoes and converts the echo delay into range. Three numbers fix its behaviour: the **bandwidth B** (how finely it separates two close targets), the **carrier / centre frequency f_c** (how far the sound travels before water absorbs it) and the **transmitted energy** (how strong the echo is compared with noise). The environment pulls these in opposite directions, which is why a fixed-waveform sonar is a compromise and a software-defined one is better.

1.1 Range from time, resolution from bandwidth

$$R = \frac{c\,\tau}{2} \qquad\qquad \delta R = \frac{c}{2B}$$

c is the speed of sound (about 1500 m/s; the Medwin approximation is c = 1449.2 + 4.6T − 0.055T² + 0.00029T³ + (1.34 − 0.010T)(S − 35) + 0.016z, T in °C, S salinity in ppt, z depth in m). τ is the two-way travel time. The factor 2 appears because sound goes out and back. **The range resolution of a pulse-compressed sonar depends only on bandwidth**, not on pulse length. So your demo's B = 25 kHz gives δR = 1500 / (2·25 000) = **3 cm**, B = 5 kHz gives **15 cm**. A real 500 kHz-class chirp with ~100 kHz bandwidth gives ~7.5 mm; you are demonstrating the same mathematics at a scaled-down frequency.

1.2 Absorption: why frequency must be chosen from the water

Seawater absorbs sound by viscosity and by the relaxation of dissolved boric acid and magnesium sulphate. The empirical Thorp formula (f in kHz, result in dB per km) is:

$$\alpha(f) = \frac{0.11 f^2}{1+f^2} + \frac{44 f^2}{4100+f^2} + 2.75\times10^{-4} f^2 + 0.003$$

Total one-way transmission loss for range R (metres) combines geometric spreading and absorption:

$$TL(R) = 20\log_{10} R + \alpha\,\frac{R}{1000}\ \ [\text{dB}]$$

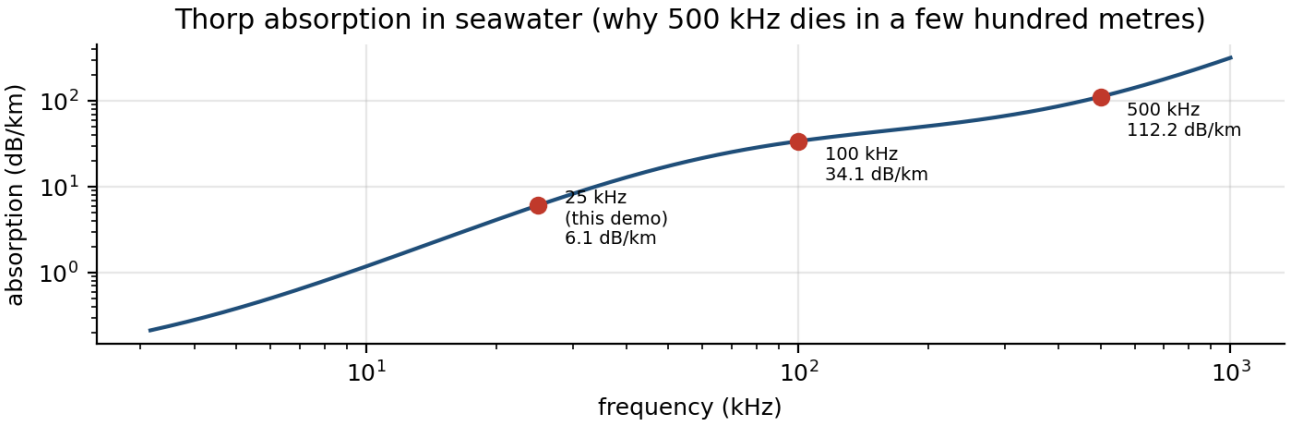


Figure 1 - Thorp absorption. 25 kHz: 6.1 dB/km; 100 kHz: 34 dB/km; 500 kHz: 112 dB/km. At 500 kHz a 300 m round trip already costs ~67 dB from absorption alone - high resolution, short reach.

1.3 The sonar equation (how energy, noise and range tie together)

$$SNR_{dB} = SL - 2\,TL + TS - NL + DI + PG$$

SL = source level (set by your **amplitude** pot and the projector), TL = one-way transmission loss, TS = target strength, NL = noise level, DI = directivity index of the receive array, and PG = **processing gain** = $10\cdot\log_{10}(B\cdot T)$ for a pulse-compressed (matched-filtered) receiver. Longer pulses and larger bandwidth both raise SNR *without* raising peak amplitude, which is the central trick of chirp sonar and why "pulse duration" is one of your three parameters.

1.4 What each pot means physically

Pot	Maps to	Muddy estuary (low)	Clear shallow reef (high)	Why
POT1 (PA1)	Bandwidth & centre freq	B = 5 kHz, f_c = 8 kHz (sweep 5.5–10.5 kHz)	B = 25 kHz, f_c = 17.5 kHz (sweep 5–30 kHz)	Turbid water scatters / absorbs high freq: use low f, narrow band, accept coarse resolution. Clear water: go wide for fine detail.
POT2 (PA2)	Pulse duration T	2 ms	20 ms	Energy ∝ $A^2 \cdot T$. Murky water needs more energy; BT product (processing gain) rises from 10 to 500.
POT3 (PA3)	Amplitude	10 % of DAC full scale	100 %	Signal power ∝ A^2. Lower it in clear/short range to save battery.

Note on the mapping: in the demo the sweep never leaves 5–30 kHz. These limits come from the hardware you have: the LM358 has a gain-bandwidth product of roughly 1 MHz and a slew rate of roughly 0.3 V/µs, which restricts large-signal output to a few tens of kHz (Section 6.3).

2. Linear Frequency Modulation (LFM) - the core mathematics

2.1 Definition

An LFM ("linear chirp") pulse of duration T sweeps linearly from f_0 to f_1. Define bandwidth $B = f_1 - f_0$, centre frequency $f_c = (f_0 + f_1)/2$ and chirp rate $k = B/T$ (Hz per second).

$$f_i(t) = f_0 + k\,t, \qquad 0 \le t \le T$$

Instantaneous frequency is the **derivative of phase divided by 2π**. Therefore the phase is the **integral** of frequency - this is the single most important idea behind every sweep:

$$\varphi(t) = 2\pi \int_0^t f_i(\xi)\,d\xi = 2\pi\left(f_0 t + \tfrac{1}{2}kt^2\right)$$

$$s(t) = A\,w(t)\sin\left[2\pi\left(f_0 t + \tfrac{1}{2}kt^2\right)\right]$$

Here *w(t)* is the amplitude window (Hann/Hamming/Blackman) and A the amplitude. In the code (`buildWaveform`, case MOD_LFM) this is literally `cyc = f0*t + 0.5f*k*t*t; s = sinCyc(cyc);` - "cyc" is the phase in *cycles* (the 2π is absorbed into the sine look-up table, Section 6.3).

Why not just write sin(2π f(t) t)? That is wrong: it makes the phase slope 2π(f + t·df/dt), i.e. twice the intended frequency sweep for a linear chirp. Frequency is d(phase)/dt, so you must integrate, not multiply.

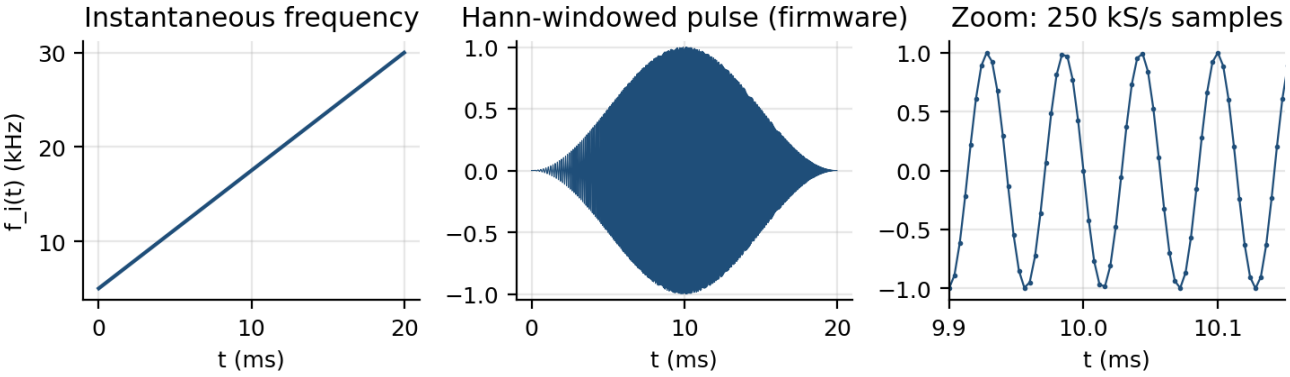


Figure 2 - A Hann-windowed 5→30 kHz, 20 ms LFM pulse computed by the firmware: instantaneous frequency (left), envelope (centre) and the individual 250 kS/s samples (right).

2.2 Time-bandwidth product and spectrum

$$TBP = B\,T$$

For large TBP (>~50) the spectrum is almost rectangular between f_0 and f_1. The reason is the **principle of stationary phase**: at frequency f the signal "spends" time $t = (f - f_0)/k$ there, so frequency and time are tied by a straight line. Energy at f therefore equals the amplitude envelope at the matching time:

$$|S(f)| \;\approx\; \frac{A}{\sqrt{2k}}\; w\!\left(\frac{f - f_0}{k}\right), \qquad f_0 < f < f_1$$

This is why a **time-domain window on an LFM pulse is also a frequency-domain window** (Figure 5): Hann amplitude in time gives a Hann-shaped bell between 5 and 30 kHz. For your largest setting (TBP = 25 kHz × 20 ms = 500) the approximation is excellent; for the smallest (5 kHz × 2 ms = 10) the spectrum shows Fresnel ripples.

2.3 Matched filter and pulse compression

The optimum linear receiver for a known waveform in white noise is the **matched filter**, whose impulse response is the time-reversed signal. Its output is the autocorrelation:

$$y(\tau) = \int s(t)\, s^*(t-\tau)\, dt, \qquad h(t) = s^*(T-t)$$

For an unweighted LFM, the compressed pulse is approximately

$$|y(\tau)| \approx T\left|\mathrm{sinc}(B\tau)\right|\left(1 - \frac{|\tau|}{T}\right)$$

- **Main-lobe width** (−3 dB) ≈ 0.886/B → the range resolution of Section 1.1. A 20 ms pulse is "compressed" to well under 0.1 ms.
- **Compression ratio and SNR gain** = B·T. In dB: $10 \cdot \log_{10}(500)$ = 27 dB for your longest wide pulse.
- **Peak sidelobe** of the unweighted sinc = −13.3 dB. This is the "ghost" echo problem; windowing (Section 4) trades main-lobe width for sidelobes of −30 dB to −58 dB.

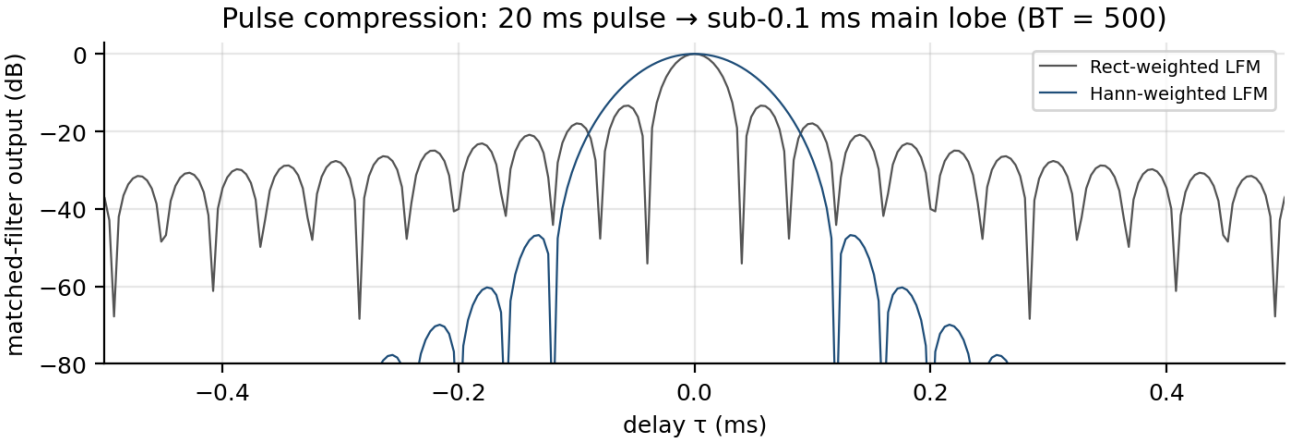


Figure 3 - Matched-filter output of the firmware pulse. Grey: rectangular weighting (−13 dB first sidelobe, sinc ringing). Blue: Hann weighting (much lower sidelobes, ~1.5× wider main lobe).

2.4 Doppler and ambiguity

A moving AUV or target shifts frequencies by $f_d = 2vf/c$. A linear chirp is "Doppler tolerant": a Doppler shift merely displaces the compressed peak in range (by f_d/k seconds) instead of destroying it. Mathematically the ambiguity function of an LFM is a ridge along the line $\tau = f_d/k$. This is the reason military and research sonars favour chirps over phase-coded pulses when platforms move.

3. Other modulations in the firmware

3.1 Geometric (exponential) sweep

Instead of adding a constant number of Hz per second, a geometric sweep **multiplies** frequency by a constant factor per second:

$$f_i(t) = f_0\, r^{\,t/T}, \quad r = \frac{f_1}{f_0}$$

Phase is again the integral. Using $\int a^x dx = a^x/\ln a$:

$$\varphi(t) = 2\pi \frac{f_0 T}{\ln r}(r^{\,t/T} - 1)$$

Firmware: `cyc = f0*Tp/lnr*(expf(lnr*t/Tp)-1)`. The sweep spends proportionally more time at low frequencies, so its spectral density falls as 1/f (a "pink" spectrum). Its advantage is that a Doppler scaling (f→αf) just becomes a time shift, so it is **Doppler-invariant**; it is used by bats and by wideband sonars with large relative bandwidth. For a narrow relative bandwidth (e.g. 5.5→10.5 kHz) it is almost identical to an LFM.

3.2 Phase-coded pulse (BPSK)

The carrier keeps a fixed frequency f_c and its phase is flipped by 0 or π according to a binary code $b_n \in \{+1, -1\}$, one code element ("chip") every T_c seconds:

$$s(t) = A\, w(t) \sum_{n=0}^{N-1} b_n \,\mathrm{rect}\!\left(\frac{t - nT_c}{T_c}\right) \sin(2\pi f_c t)$$

Bandwidth is set by the *chip rate* (null-to-null width ≈ $2/T_c$). The firmware uses $T_c = 2/B$ so that the main lobe fits inside your chosen bandwidth, giving $N = B{\cdot}T/2$ chips (range 13 … 511). The matched-filter output is the code autocorrelation

$$R[m] = \sum_n b_n\, b_{n+m}$$

- **Barker-13** (+++++−−++−+−+): R[0] = 13 and every |R[m]| ≤ 1, a sidelobe ratio of $20{\cdot}\log_{10}(1/13) = -22.3$ dB. The firmware uses it when N ≤ 13 (shortest, narrowest pulses).
- **Maximal-length sequence** (LFSR, x^9+x^5+1, period $2^9-1 = 511$): periodic autocorrelation is exactly −1 at every non-zero lag (peak/sidelobe = 511). The firmware sends the first N chips (a truncated m-sequence - the sidelobes become noise-like instead of exactly −1, Figure 4).
- **Compression gain** = N ($10{\cdot}\log_{10}$N dB), the same B·T idea as the chirp.
- Trade-off: excellent sidelobe/range properties but **not Doppler tolerant**; a Doppler shift of even a fraction of the chip rate de-correlates the code.

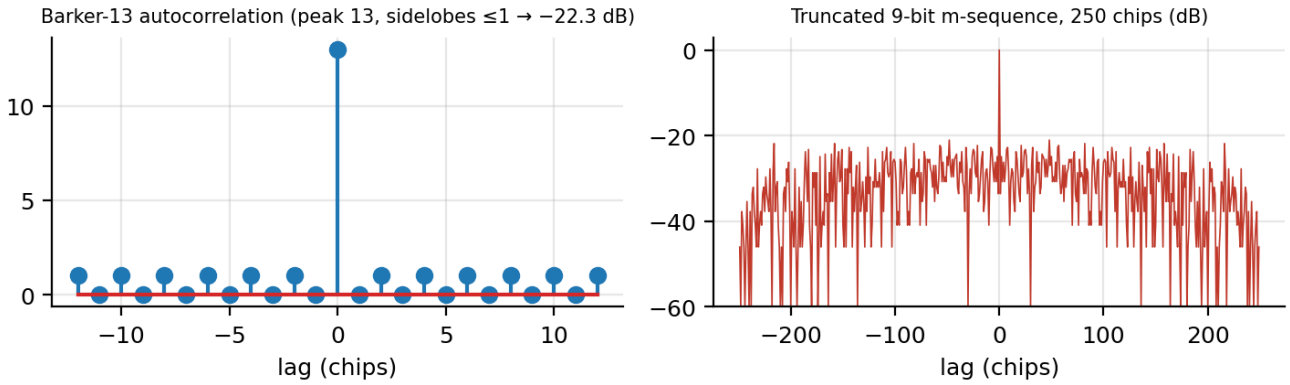


Figure 4 - Left: Barker-13 autocorrelation. Right: autocorrelation (dB) of the 250-chip truncated m-sequence used for wide, long pulses.

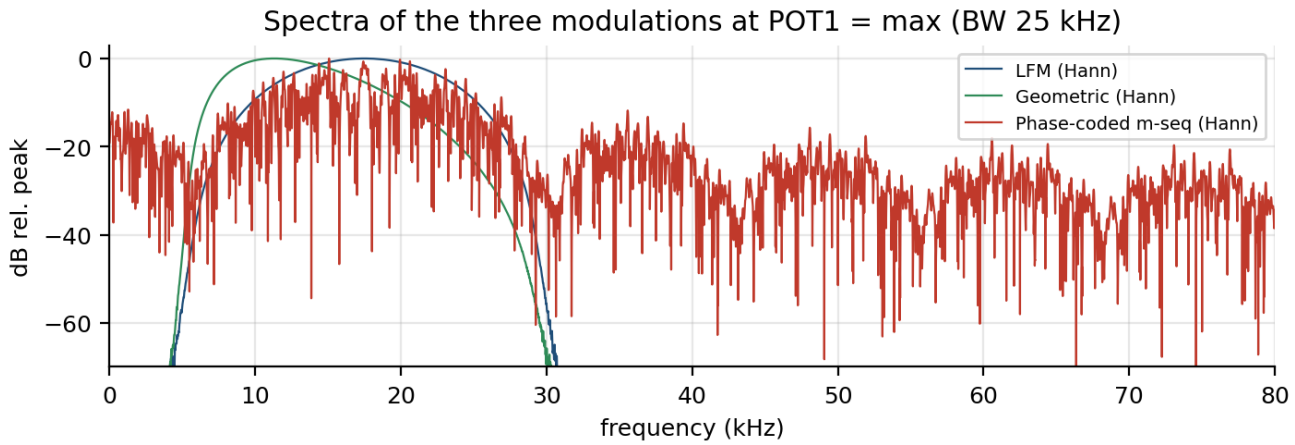


Figure 5 - Spectra computed from the three firmware waveforms at POT1 = max. LFM and geometric fill 5–30 kHz; the BPSK spectrum is a wide sinc² shape - phase-coded pulses leak more energy outside the band, so the analog low-pass filter matters most in this mode.

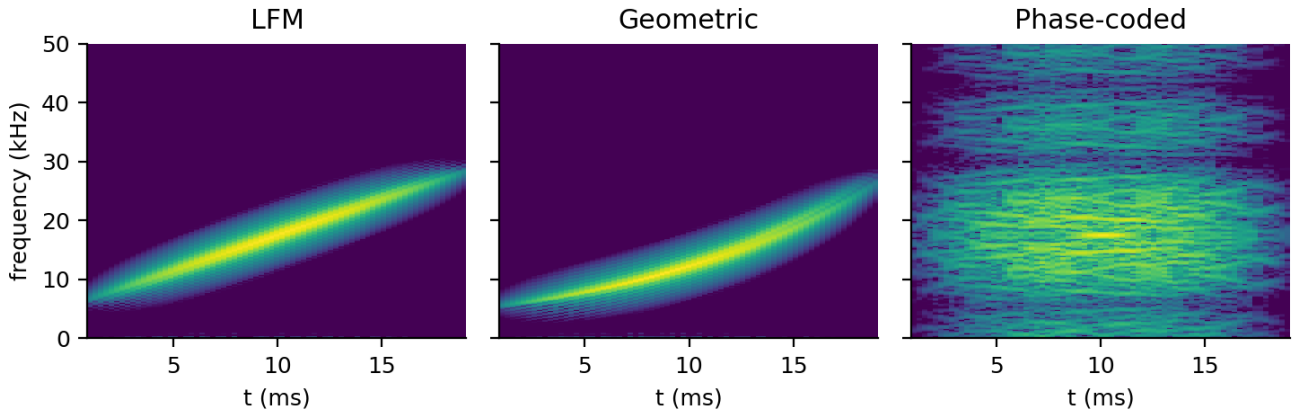


Figure 6 - Spectrograms (time vs. frequency). LFM: a straight diagonal. Geometric: a curve that rises slowly then quickly. Phase-coded: a constant tone at f_c with broadband "flash" at each phase flip.

4. Windowing (apodisation)

Starting or stopping a sine abruptly is multiplication by a rectangle in time, i.e. convolution with a sinc in frequency - the sinc's sidelobes (−13 dB, decaying only 6 dB/octave) smear energy across the whole spectrum ("spectral splatter") and produce voltage steps that stress the transmitter and the projector. A smooth window w(m) forces the pulse to start and end at zero:

$$w_{\text{Hann}}[m] = 0.5 - 0.5\cos\frac{2\pi m}{M-1} \qquad w_{\text{Hamming}}[m] = 0.54 - 0.46\cos\frac{2\pi m}{M-1}$$

$$w_{\text{Blackman}}[m] = 0.42 - 0.5\cos\frac{2\pi m}{M-1} + 0.08\cos\frac{4\pi m}{M-1} \qquad m = 0, \ldots, M-1$$

Window	Peak sidelobe	Side-lobe decay	Main-lobe width (null-null, bins)	Coherent gain	Mean-square (power) factor	ENBW (bins)	Start/end value
Rectangular	−13.3 dB	6 dB/oct	2	1.00	1.000	1.00	1 (abrupt)
Hann	−31.5 dB	18 dB/oct	4	0.50	0.375	1.50	0
Hamming	≈ −43 dB	6 dB/oct	4	0.54	0.397	1.36	0.08
Blackman	−58 dB	18 dB/oct	6	0.42	0.305	1.73	0

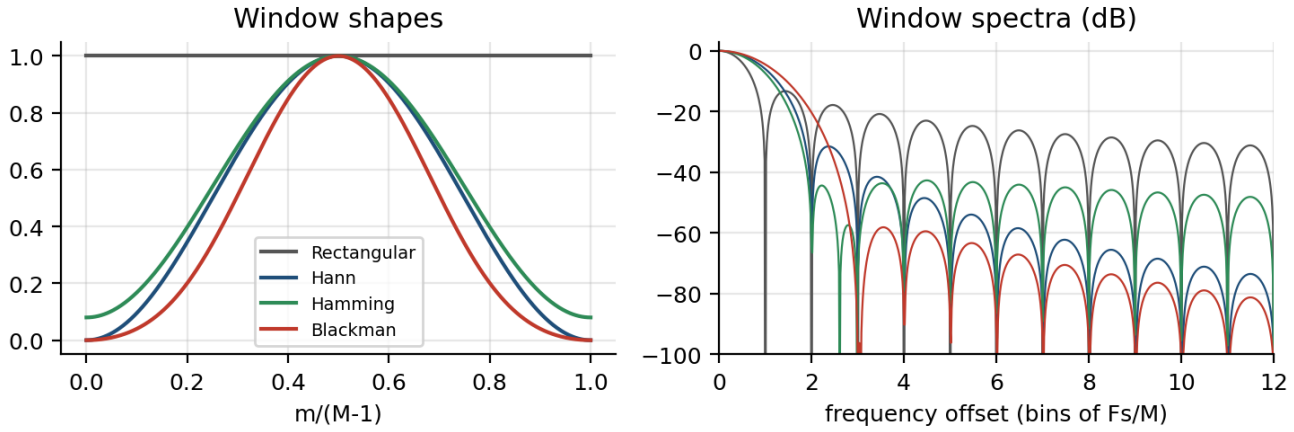


Figure 7 - Left: window shapes. Right: their frequency responses; more sidelobe suppression costs a wider main lobe.

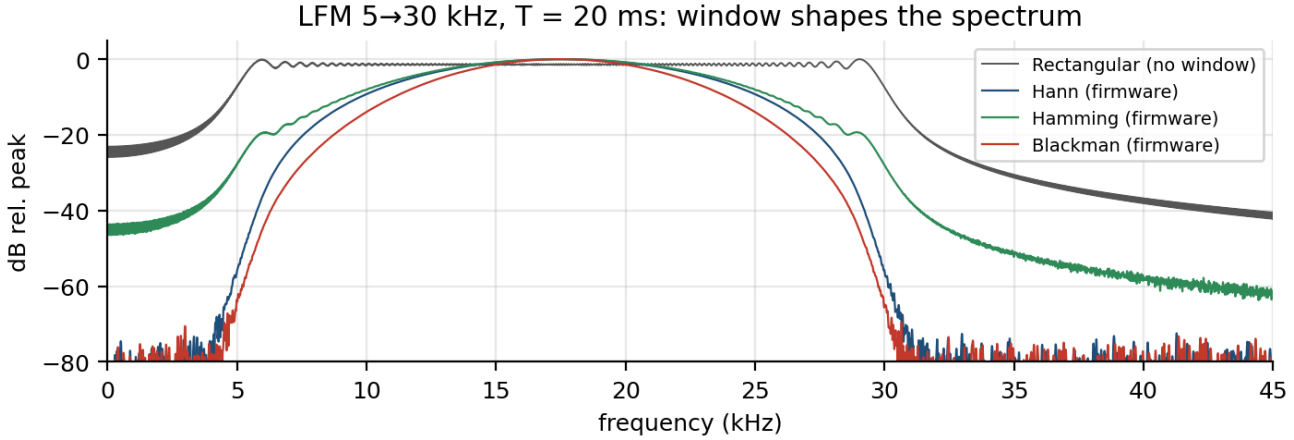


Figure 8 - Spectrum of the 20 ms LFM from the firmware with each window. Because frequency is mapped to time in a chirp, the window's shape shows up directly as the in-band shape; the out-of-band skirts drop much faster for Blackman/Hann than for the rectangular pulse.

Design consequences. (1) Windowing on transmit reduces average power by the mean-square factor above (Hann: only 37.5 % of the unwindowed power for the same peak amplitude) - there is no free lunch.

(2) Hamming ends at 0.08, not 0, so a small step remains; the firmware pads with mid-scale samples and the analog low-pass filter handles the rest. (3) LFM pulses can instead be left rectangular on transmit and weighted on receive, which keeps energy but costs matched-filter loss. Transmit-side windowing is chosen here because the problem statement asks to protect the transmitter and remove sidelobe artefacts.

5. Sampling, quantisation and the DAC

5.1 Sample rate choice

Nyquist requires $F_s > 2 \cdot f_{max}$. With f_{max} = 30 kHz the minimum is 60 kHz, but an *analog* reconstruction filter needs room between the signal and the first image. The firmware uses F_s = 250 kHz:

$$F_s = \frac{f_{TIM1}}{(PSC+1)(ARR+1)} = \frac{100\,\text{MHz}}{1 \cdot 400} = 250\ \text{kHz}, \quad T_s = 4\,\mu\text{s}$$

The first image of a 30 kHz tone sits at $F_s - 30$ kHz = 220 kHz, a factor of 7.3 above the highest signal frequency, so even a 2-pole filter attenuates it by ~26 dB (Figure 9). 250 kHz also gives 8.3 samples per cycle at 30 kHz, so the staircase output is smooth.

5.2 Quantisation noise

$$LSB = \frac{V_{ref}}{2^N} = \frac{3.3\,V}{4096} = 0.806\,\text{mV}, \qquad SQNR \approx 6.02N + 1.76 = 74\ \text{dB}\ (N = 12)$$

SQNR is for a full-scale sine. At 10 % amplitude (POT3 minimum) the signal is 20 dB smaller, so SQNR drops to ~54 dB - one reason not to leave the amplitude pot low when you want clean FFT pictures. Because the signal is windowed, only the middle of the pulse reaches full amplitude.

5.3 Zero-order hold and images

A DAC holds each sample for T_s, equivalent to filtering the ideal impulse train by a rectangular pulse. Its frequency response is

$$H_{ZOH}(f) = \text{sinc}\left(\frac{f}{F_s}\right) = \frac{\sin(\pi f/F_s)}{\pi f/F_s}$$

At 30 kHz: sin(0.377)/0.377 = 0.976, i.e. −0.21 dB droop - negligible. Spectral images appear around every multiple of F_s (at $kF_s \pm f$) and must be removed by the low-pass filter (Section 6). The sinc alone attenuates the first image (220 kHz for a 30 kHz tone) by only sinc(0.88) = 0.133, i.e. about −17.5 dB, so the analog filter does most of the work; the two attenuations add in dB.

5.4 DDS view and the sine look-up table

The firmware is a software Direct Digital Synthesiser: a **phase** in cycles is converted to amplitude by a 1024-point sine table with linear interpolation between neighbouring entries:

$$idx = \{\phi\} \cdot 1024, \quad i = \lfloor idx \rfloor, \quad \sin(2\pi\phi) \approx L[i] + (idx - i)\,(L[i+1] - L[i])$$

The worst-case interpolation error of a sine is $(\Delta\theta)^2/8$ with $\Delta\theta = 2\pi/1024 = 0.00614$ rad, giving 4.7×10^{-6} (−106 dB) - far below the 12-bit quantisation floor (2.4×10^{-4}). Building the waveform with a table costs only a few multiplies per sample instead of a full `sinf()`, and the Cortex-M4 FPU does these in a handful of cycles. {φ} means the fractional part, so the phase wraps automatically at every full cycle (this is exactly what an FPGA phase accumulator does with integer overflow).

6. The analog front-end: Sallen-Key low-pass (LM358)

6.1 Why it is needed

- **Removes DAC images** and the stair-step ripple at F_s.
- **Smooths the edges** the DMA/SPI chain creates (code steps, glitches at CS rising edges).
- **Limits the spectrum** so no power is wasted above f_1, protecting the class-D amplifier (TPA3116D2) and projector.

6.2 Unity-gain Sallen-Key (2nd order)

Two resistors in series (R_1, R_2), a capacitor C_2 from the op-amp's + input to ground, and a capacitor C_1 from the junction of the resistors to the op-amp output (the positive feedback path). The transfer function is

$$H(s) = \frac{\omega_0^2}{s^2 + \frac{\omega_0}{Q}s + \omega_0^2}, \qquad \omega_0 = \frac{1}{\sqrt{R_1 R_2 C_1 C_2}}, \qquad Q = \frac{\sqrt{R_1 R_2 C_1 C_2}}{C_2 (R_1 + R_2)}$$

For equal resistors Q = ½·√(C_1/C_2). A **Butterworth** (maximally flat) response needs Q = 0.707, i.e. C_1 = 2·C_2. Roll-off is 40 dB/decade above f_0; |H| at frequency f is

$$|H(f)| = \frac{1}{\sqrt{(1 - (f/f_0)^2)^2 + (f/(Qf_0))^2}}$$

Example (not necessarily your values): R = 2.2 kΩ, C_1 = 2.2 nF, C_2 = 1 nF gives f_0 = 48.8 kHz, Q = 0.74, a droop of only −0.31 dB at 30 kHz and −26 dB at 220 kHz. Compute the f_0 of *your* component values with the formula above; to keep a flat 5–30 kHz passband it should be roughly 40–60 kHz.

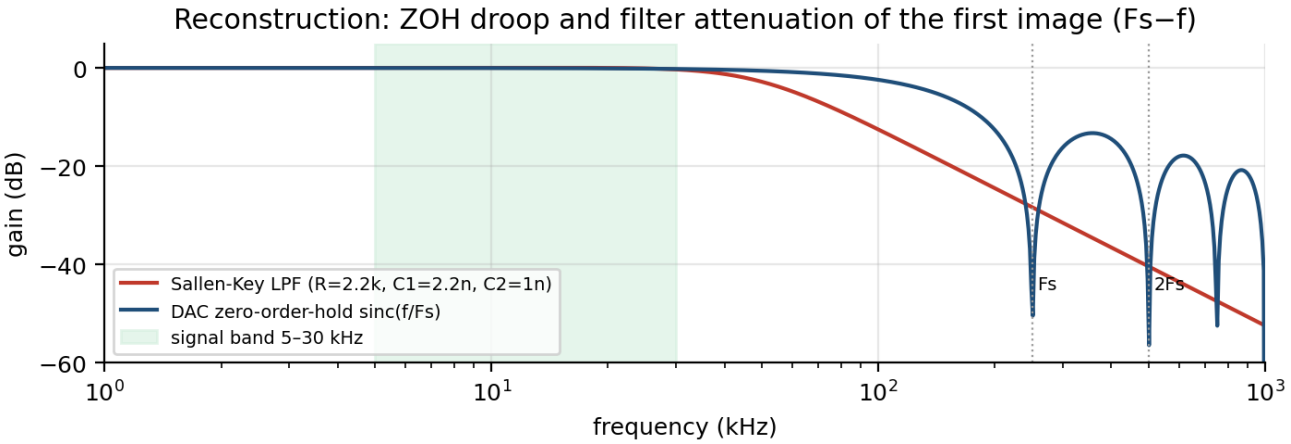


Figure 9 - Example Sallen-Key response and the zero-order-hold droop, with the 5–30 kHz signal band shaded and F_s, $2F_s$ marked.

6.3 LM358 on a single 5 V supply - practical limits

- **Slew rate.** A sine of peak V_p needs slew = 2πfV_p. For V_p = 1.65 V at 30 kHz that is 0.31 V/µs - right at the LM358's ≈0.3 V/µs limit. The full-power bandwidth f_{max} = SR/(2πV_p) ≈ 29 kHz. If the top of the sweep looks like a triangle on the scope, reduce POT3 (amplitude) or the upper frequency limit.
- **Output swing.** It reaches only about V+ − 1.5 V (≈ 3.5 V) on a 5 V rail; the DAC's 0–3.3 V signal fits, but anything gained above unity would clip.
- **Crossover distortion.** The LM358 output stage has no quiescent current in its pull-down; a 5–10 kΩ resistor from the output to ground improves symmetry for a single supply.
- **DC level.** The DAC idles at mid-scale (1.65 V). The coupling capacitor to the amplifier removes this DC; a 10 µF–100 µF capacitor in series with the TPA3116D2 input network keeps the low-frequency corner far below 5 kHz.

7. Timer + DMA architecture (the "hardware-level optimisation")

7.1 What DMA is

A **Direct Memory Access** controller is a small bus-master that copies data between memory and peripherals without executing instructions. The STM32F411 has two DMA controllers with 8 *streams* each. Each stream has a **channel** multiplexer (0–7) that selects which peripheral's request line triggers it. A transfer is configured by: source address, destination address, number of items (NDTR), data width, increment flags, direction, priority and mode (normal = stop at zero, circular = reload automatically).

Register / field	Meaning in this project
CHSEL	Request channel. We use channel 6 on DMA2 so that the timer TIM1 requests reach streams 1, 4, 5; channel 0 on stream 0 for ADC1.
DIR = memory→peripheral	Waveform words go to SPI1->DR; constants go to GPIOA->BSRR.
MINC	Memory increment ON for the waveform buffer (step through samples); OFF for the single constant that sets/clears CS.
PSIZE / MSIZE	16-bit for SPI data; 32-bit for BSRR (set/reset bits live in one 32-bit register).
PL (priority)	High / very high. Timing is deterministic because only one transfer happens at any instant.
NDTR	Number of samples in the pulse; the TCIF flag raises when it reaches zero.
Mode	Normal (one-shot per ping) - after the pulse the streams stop by themselves.

7.2 Why one stream is not enough: the MCP4921 chip-select problem

The MCP4921 loads its 16-bit command word while CS is low and transfers it to the DAC output on CS's **rising edge**. A DMA stream feeding SPI1->DR would stream words back-to-back, but CS would not toggle between them. STM32F4 SPI cannot automatically pulse NSS between frames (the NSS-pulse mode exists on newer families). The solution: **one timer, three DMA requests per sample**, each driving a different destination.

TIM1 event	Time in sample	DMA2 stream / ch	Writes	Effect
Update (counter reload)	0 µs (0 %)	Stream 5 / ch 6	GPIOA->BSRR = 1<<20	CS (PA4) falls
Capture/Compare 1	0.6 µs (15 %)	Stream 1 / ch 6	SPI1->DR = txBuf[n++]	16 bits clocked out in 1.28 µs
Capture/Compare 4	2.4 µs (60 %)	Stream 4 / ch 6	GPIOA->BSRR = 1<<4	CS rises → DAC output updates (LDAC = 0)

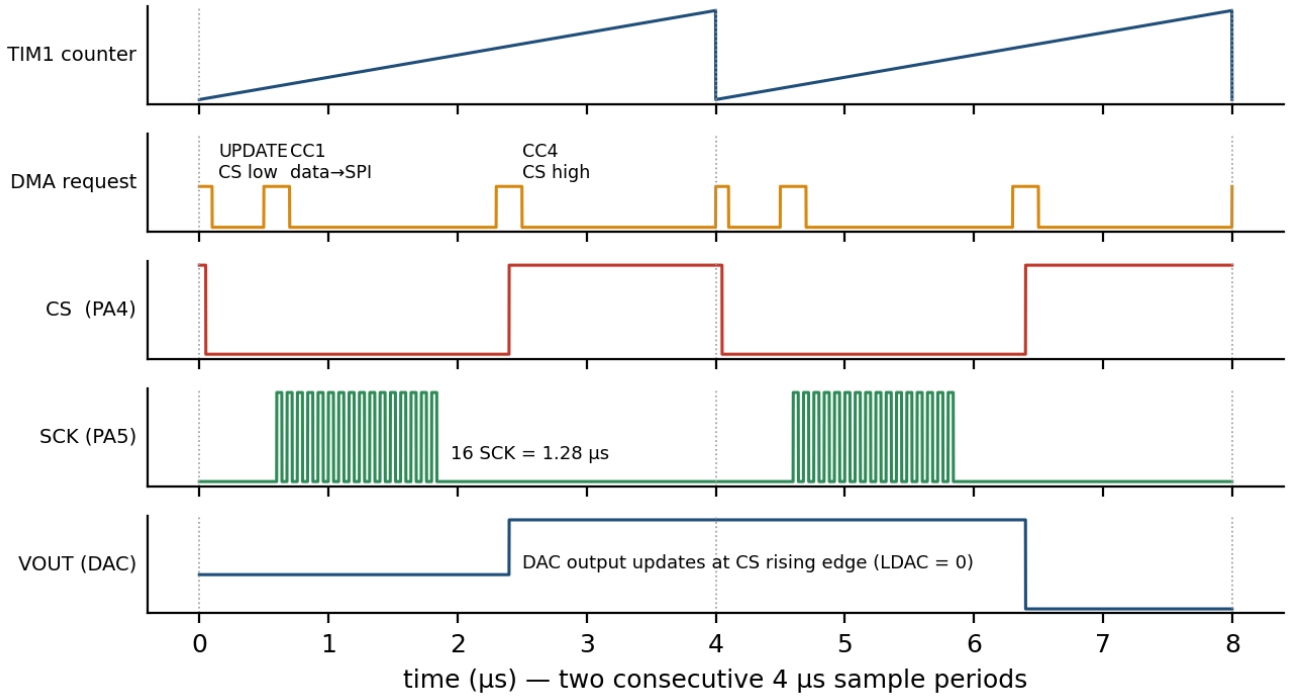


Figure 10 - Per-sample choreography. TIM1 produces three DMA requests; the SPI frame (16 clocks at 12.5 MHz = 1.28 µs) finishes well before the CS rising edge at 60 % of the period. The CPU is not involved.

The BSRR register. GPIO pins are set/reset by writing to the Bit Set/Reset Register: bits 0–15 *set* pins, bits 16–31 *reset* pins. A single 32-bit write is atomic - no read-modify-write - which is why DMA can safely toggle one pin.

7.3 Timing arithmetic and bus load

$$T_{SPI,\,frame} = \frac{16}{f_{SCK}} = \frac{16}{12.5\,\text{MHz}} = 1.28\,\mu\text{s} < 0.45\,T_s = 1.8\,\mu\text{s}$$

Constraints from the MCP4921 datasheet: $f_{SCK} \le 20$ MHz, a few tens of nanoseconds between CS falling and the first clock, and ≥ 15 ns between the last clock and CS rising - the timer offsets give hundreds of nanoseconds of margin on each. DMA load: 3 transfers × 250 k samples/s = 750 000 transfers/s on a 100 MHz bus - far below 1 % utilisation. The CPU executes `__WFI()` (wait-for-interrupt) in the main loop: the core clock is gated while DMA and timers keep running (Sleep mode). It wakes on the 1 ms SysTick to check a timeout, then sleeps again.

Check: the MCP4921 output buffer settles in about 4.5 µs and slews at roughly 0.55 V/µs (datasheet values: verify against your revision). At 4 µs per sample and 30 kHz the largest step is $2\pi f V_p / F_s \approx 1.2$ V, comfortably handled because consecutive samples of a sine differ by much less than full scale.

7.4 Ping-pong scheduling (what the CPU does and when)

Phase	Duration	Who works	Notes
Sleep until ping time	~80–98 ms	Nobody (WFI)	Button polled each 1 ms tick
ADC scan (3 ch × 8 averages)	~0.15 ms	ADC + DMA2 S0	CPU busy-waits only for a few 100 µs
Waveform synthesis	a few ms (LUT interpolation)	CPU (FPU)	Computes up to 5000 windowed samples
Transmit	T = 2–20 ms	TIM1 + 3 DMA streams + SPI	CPU in WFI

The duty cycle of CPU-active time is therefore only a few percent, and TX time is T/PRI = 2–20 %. A possible further optimisation (not needed for the demo) is double-buffering: compute the next waveform while the current one plays, using the second half of RAM.

8. Reading the "sensors": ADC + DMA

Each pot is a ratiometric divider: V_{wiper} = x·3.3 V, x ∈ [0,1]. The ADC returns a 12-bit code D = round(4095·x). The firmware:

- Configures ADC1 in **scan mode** over channels IN1, IN2, IN3 (PA1, PA2, PA3).
- Uses a DMA stream (DMA2 Stream 0, channel 0) in **peripheral→memory** mode to collect ADC_AVG = 8 scans (24 half-words) without CPU intervention.
- Averages the 8 values per channel. For uncorrelated noise, averaging N samples improves SNR by √N → 9 dB for N = 8.
- Turns the ADC off after the burst - a powered ADC draws current even when idle.

$$t_{conv} = \frac{t_{sample} + 12}{f_{ADC}} = \frac{144 + 12}{25\,\text{MHz}} = 6.2\ \mu\text{s} \ \text{ per channel}, \qquad 24 \times 6.2\,\mu s \approx 0.15\ \text{ms}$$

The sampling time (144 cycles) must be long enough for the sample capacitor to charge through the pot's source resistance (≤ 2.5 kΩ for a 10 kΩ pot at the centre) - a rule of thumb is $R_{source} < t_{sample}/(f_{ADC} \cdot C_{ADC} \cdot \ln 2^{13})$. A 100 nF capacitor from each wiper to ground helps.

Linear parameter mapping used by the adaptation logic (x_1, x_2, x_3 are the three normalised pot values):

$$B = 5 + 20x_1\ \text{kHz}, \quad f_c = 8 + 9.5x_1\ \text{kHz}, \quad T = 2 + 18x_2\ \text{ms}, \quad A = 0.1 + 0.9x_3$$

$$f_0 = f_c - \frac{B}{2}\ (5.0\ldots5.5\ \text{kHz}), \qquad f_1 = f_c + \frac{B}{2}\ (10.5\ldots30\ \text{kHz})$$

Because the three parameters are re-read before *every* ping (every 100 ms), a change of a pot becomes a different waveform on the very next ping - the "instant" adaptation required by the problem statement.

9. Energy, power and battery

The acoustic energy of one pulse is the integral of the squared (windowed) signal:

$$E = \frac{1}{Z}\int_0^T s^2(t)\,dt = \frac{A^2 T}{2Z} \cdot \overline{w^2}$$

with the mean-square window factor \overline{w²} = 0.375 (Hann), 0.397 (Hamming), 0.305 (Blackman), 1 (rectangular) and Z the acoustic impedance.

Energy scales with A² and linearly with T - which is why the two pots "amplitude" and "pulse duration" are both energy controls. A long, low-amplitude chirp can deliver the same energy as a short, high-amplitude one but with far lower peak voltage (less electrical stress) and the same bandwidth-limited resolution thanks to pulse compression. The average transmit-electronics power is

$$P_{avg} = P_{idle} + \frac{T}{PRI}\,(P_{TX} - P_{idle})$$

Techniques used: (1) CPU in WFI while DMA runs; (2) ADC powered only during a 0.15 ms burst; (3) a short pulse at low amplitude when the environment allows; (4) DAC parked at mid-scale between pings so there is no step into the AC-coupled amplifier (shutting the DAC down would drop its output to 0 V and send a "thump" through the coupling capacitor); (5) optional: lower the system clock or switch off the LED. Measure the supply current with a multimeter in series to quantify the savings for your judges.

10. Validating the output with an oscilloscope FFT

What to expect

- **Time domain:** a bell-shaped envelope with the carrier frequency visibly increasing along the pulse; the start and end are smooth (Hann/Blackman) with no jump. A new ping every 100 ms - set the scope to normal trigger on the rising edge or use single-sequence mode.
- **FFT:** choose a time window that contains the whole pulse (e.g. 10 ms/div for a 20 ms pulse) - then the FFT is the pulse spectrum; a shorter window shows only a part of the sweep. Frequency resolution $\Delta f = F_{s,scope}/N_{FFT} = 1/T_{window}$; for 100 ms of data that is 10 Hz per bin.
- **Spectrum shape:** a flat-topped band between f_0 and f_1 shaped like the chosen window (Figure 8) for LFM; for short pulses (small time-bandwidth product) you will see ripples.
- **Scope's own FFT window:** choose Hann or Flat-top in the scope. A rectangular scope window adds its own sidelobes and hides the benefit of your transmit window.
- **Out-of-band floor:** −50…−60 dB is a good result for a 12-bit DAC and an op-amp on a breadboard. Mains pickup (50 Hz harmonics) and breadboard noise appear as low-frequency spurs.

Debug checklist

Symptom	Likely cause
No output at all, LED blinks at 10 Hz	Check LDAC (PB0) is LOW, MCP4921 VDD/VREF, CS (PA4) toggling with a logic probe; DAC output must idle at 1.65 V.
Output stuck at mid-scale; LED flashes	DMA request mapping or clocks wrong: enable `DEBUG_SERIAL` and look for DMA-TIMEOUT.
Staircase visible on the scope	Reconstruction filter cut-off too high or missing; verify f_0 with the Sallen-Key formula.
Top of the sweep distorted / triangular	LM358 slew limit; lower POT3 or POT1 maximum. Add pull-down resistor on the output.
Low-frequency "thump" each ping	Pulse not starting/ending at mid-scale; use Hann or Blackman; check coupling capacitor value.
Window mode seems to change with button noise	Debounce: short press > 30 ms; long press ≥ 0.8 s.

11. Code-to-math map

Function in SonarTx.ino	Mathematics / concept	Section
`initTables()`	Sine LUT: $L[i] = \sin(2\pi i/1024)$; LFSR m-sequence x^9+x^5+1	5.4, 3.2
`sinCyc()`	Phase in cycles → amplitude, linear interpolation	5.4
`windowFn()`	Hann/Hamming/Blackman formulas	4
`readPots()`	ADC scan + DMA + averaging	8
`mapParams()`	B, f_c, f_0, f_1, T, A from pot ratios	8
`buildWaveform()`	LFM $\varphi = f_0 t + kt^2/2$; geometric $\varphi = f_0 T(r^{t/T}-1)/\ln r$; BPSK $\pm\sin(2\pi f_c t)$	2, 3
`transmit()`	TIM1 + DMA2 S5/S1/S4, CS choreography, WFI sleep	7
`hwInit()`	SPI1 12.5 MHz, 16-bit; TIM1 ARR = $f_{tim}/F_s - 1$; ADC clock; GPIO	5, 7, 8

12. Self-test question bank

Work through these without looking back; the answers follow.

1. A linear chirp sweeps from 5 kHz to 30 kHz in 20 ms. Compute k, B, f_c, TBP, the unweighted range resolution at c = 1500 m/s and the pulse-compression gain in dB.
2. Why is the phase of a chirp $2\pi(f_0 t + \tfrac{1}{2}kt^2)$ and not $2\pi(f_0 + kt)t$?
3. With F_s = 250 kHz, what are the timer ARR value and the CCR1/CCR4 values for a 100 MHz timer clock? What is the SPI frame time at fPCLK2/8?
4. Why do we need three DMA streams per sample? What happens to the DAC output if CS stays low?
5. Why is the Hann window better than the rectangular window for sidelobes, and what do you pay for it?
6. Calculate the −0.2 dB droop of the DAC at 30 kHz. Where is the first image and how much does a 2-pole Butterworth at 48.8 kHz attenuate it?
7. What is the LM358 full-power bandwidth at 1.65 V peak if SR = 0.3 V/µs?
8. A Barker-13 code is received: what is the peak-to-sidelobe ratio in dB? What is the compression gain?
9. Explain why a pot change affects the next ping but never the middle of a ping.
10. Estimate the energy ratio of a 20 ms Hann pulse at A = 1 to a 2 ms Hann pulse at A = 1. What amplitude does the short pulse need to match the energy of the long pulse?

Answers

- **1.** k = 25 kHz / 0.02 s = 1.25 MHz/s; B = 25 kHz; f_c = 17.5 kHz; TBP = 500; $\delta R = 1500/(2\cdot 25000) = 3$ cm; gain = $10\cdot\log_{10}500 = 27$ dB.
- **2.** Frequency is the derivative of phase / 2π. $d/dt[2\pi(f_0 t + \tfrac{1}{2}kt^2)] = 2\pi(f_0 + kt) \rightarrow f_0 + kt$. The wrong form differentiates to $2\pi(f_0 + 2kt)$ - twice the sweep.
- **3.** ARR = 100 MHz/250 kHz − 1 = 399; CCR1 = 60, CCR4 = 240; frame = 16/12.5 MHz = 1.28 µs.
- **4.** The DAC latches on CS rising edge; without toggling CS nothing would ever be transferred to the output. One stream drives CS low, one sends the data, one drives CS high; a single stream can have only one destination address.
- **5.** It has a smooth tail (−31.5 dB first sidelobe, 18 dB/oct decay vs −13.3 dB, 6 dB/oct) at the price of a 2× wider main lobe (4 bins vs 2), 1.5× ENBW and 62.5 % power loss.
- **6.** sinc(0.12) = 0.976 → −0.21 dB; first image at 220 kHz; filter |H| ≈ −26 dB plus ZOH sinc ≈ −17.5 dB → ≈ −43 dB total.
- **7.** $SR/(2\pi V_p) = 0.3\times 10^6/(2\pi\cdot 1.65) = 28.9$ kHz.
- **8.** $20\cdot\log_{10}(1/13) = -22.3$ dB peak-to-sidelobe; compression gain $10\cdot\log_{10}13 = 11.1$ dB.
- **9.** The ADC is read and the buffer rebuilt *before* the pulse starts; the DMA plays that fixed buffer unchanged.
- **10.** Energy ∝ A^2T, so the ratio is 10 : 1. To match it the short pulse would need $A = \sqrt{10} \approx 3.16$ times larger - impossible if the long pulse is already at DAC full scale, which is why pulse duration is a genuine energy control and not just a resolution setting.

13. Further reading and datasheets

- M. Skolnik, *Radar Handbook* - chapters on pulse compression (LFM, Barker, m-sequences); the mathematics is the same for sonar.
- R. J. Urick, *Principles of Underwater Sound* - sonar equation, Thorp absorption.
- C. E. Cook and M. Bernfeld, *Radar Signals* - ambiguity functions.
- A. V. Oppenheim and R. Schafer, *Discrete-Time Signal Processing* - windows, sampling, reconstruction.
- F. J. Harris, "On the Use of Windows for Harmonic Analysis with the DFT," Proc. IEEE, 1978.
- ST RM0383 (STM32F411 reference manual): DMA (chapter 9, request mapping table), TIM1 (chapter 12), SPI (chapter 20), ADC (chapter 11).
- Microchip MCP4921/4922 datasheet (DS21897); TI LM358 datasheet; TI TPA3116D2 datasheet.

- Texas Instruments application note SLOA024 "Active low-pass filter design" for Sallen-Key.